

Szoftverfejlesztés Mérnököknek – JPA gyakorlat

Ebben a projektben a [JPA](#) használatát gyakoroljuk [Hibernate](#) megvalósításon keresztül

1) Indítsunk egy új projektet maven archetype-ból

a) Netbeans

- Válasszuk az alábbi lehetőséget: File -> New project -> Java with Maven -> Project from Archetype
- keressük ki a “jpa-hibernate-playground” archetype-ot, vagy
- Használjuk az alábbi maven koordinátákat:
Group ID: io.github.dennysfredericci
Artifact ID: jpa-hibernate-playground
Version: 1.0.0
Repository: https://repo.maven.apache.org/maven2/
- Állítsuk be a project nevét tetszőlegesen, majd Finish...

b) IntelliJ IDEA

- Válasszuk az alábbi lehetőséget: File -> New Project -> Maven
- Jelöljük ki a “Create from archetype” checkbox-ot, majd “Add Archetype...”
- Írjuk be az 1/a) pont alatt megadott maven koordinátákat, majd “OK”
- Válasszuk az újonnan hozzáadott archetype-ot, majd “Next”
- Válasszunk projekt nevet tetszés szerint, majd (Next-next...) Finish...

2) Vizsgáljuk meg a pom.xml fájlt

- a) A pom.xml fájlban a JavaSDK-dnak megfelelő source és target értékeket kell látnod, de legalább 11-es
- b) Vegyük észre a függőségek között a com.h2database.h2 valamint a org.hibernate.hibernate-core függőségeket

3) Állítsuk be az adatbázist

- a) Vizsgáljuk meg a persistence.xml fájlt. Itt állíthatjuk be, hogy milyen módon érjük el az adatbázist..

- A JDBC URL az alábbiak szerint van beállítva:

```
<property name="javax.persistence.jdbc.url" value="jdbc:h2:mem:my_database"/>
```

- Eszerint az adatbáziskezelő rendszer “in-memory” módban fut, azaz minden alkalommal, amikor az alkalmazást elindítod, újra fogja építeni az adatbázist, a leállításkor pedig törlődik minden adat. Hogy igazi perzisztenciát valósíthassunk meg, módosítsuk ezt a beállítást az alábbiak szerint:

```
<property name="javax.persistence.jdbc.url" value="jdbc:h2:file:~/my_database"/>
```

ahol a my_database egy általad választott adatbázis név kell, hogy legyen. Eredményképpen az adatbázis a saját könyvtáradban tárolódik majd egy my_database.mv.db nevű fájlban, ahol a név eleje illeszkedni fog az általad fenn megadott adatbázisnévre.

- Opcionálisan állítsuk be a felhasználónevet és jelszót is

b) Tekintsük át a további sorokat, azok jelentését

- `<property name="hibernate.hbm2ddl.auto" value="create"/>`

Módosítsuk ezt a sort `validate` értékre, hogy a táblákat ne generálja mindig újra a rendszer az alkalmazás indulásakor.

4) Build, majd futtatás

- NetBeans esetén: Clean and Build, Run
- IntelliJ IDEA -> Build Project  (CTRL-F9), majd jobbklikk a main metódust tartalmazó osztály nevén és Run (Application.main())

5) A program futása közben a böngészőben elérhető a <http://localhost:8082/> címen a H2 adatbáziskezelő web interfésze.

- Használjuk a 3/a) pontban megadott információkat az adatbázisba való belépéshez, majd tekintsük át annak tartalmát.

6) Nézzük meg Customer osztály példáján keresztül, hogy épül fel egy minimális JPA entitás

7) Készítsünk saját ORM leképezést az alábbiak szerint egy Animal osztályhoz

8) Készítsük el az osztályt a következő attribútumokkal:

- gender (enum MALE/FEMALE),
- age (int),
- name (String),
- id (int)

Adjunk az osztály attribútumaihoz getter és setter metódusokat, valamint esetleg egy alapértelmezett konstruktort. (Ezt nem mindig követeli meg kötelezően a Jpa, de nem baj, ha van)

a) Netbeans

- Készítsük el az osztályt és annotáljuk kézzel
- Az osztály neve legyen `Animal`, a csomag legyen azonos a `Customer` osztálynál használttal
- Use persistence annotations to create the animal entity

(1) Use `@Entity` and `@Table(name = "animal")` on the `Animal` class

(2) Use `@Id`, and `@GeneratedValue(strategy = GenerationType.AUTO)` on the `id` field

(3) Use `@Column(name = "...", unique = ... , nullable = false)` on all fields. Set unique to true for the id, and for false for the other fields.

b) IntelliJ IDEA Ultimate

- Válasszuk a View -> Tool Windows -> Persistence lehetőséget

- Jobbklikk a perzisztencia egység nevén (br.com.fredericci.pu by default), majd válasszuk a New -> Entity-t
- Jobbklikk a az Animal entitás nevén és adjunk meg új @Basic típusú elemeket az entitáshoz. Ezek lesznek az osztály attribútumai (New -> Basic).

10) Adjuk a `<class>kocsisg.jpatest_netbeans.Animal</class>` sort a `persistence.xml` fájlhoz közvetlenül a Customer alá (IntelliJ esetén automatikus).

11) példányosítsunk egy új Animal objektumot és mentjük az adatbázisba

a) Példányosítsuk az Animalt .

b) Mentés az adatbázisba:

```
entityManager.getTransaction().begin();  
entityManager.persist(customer);  
entityManager.persist(elephant);  
entityManager.getTransaction().commit();
```

Figyeljük meg az adatbázisban megjelenő új adatokat

12) Hogy a gender értékét szám helyett szöveggel mentjük az adatbázisban, használjuk az `@Enumerated(EnumType.STRING)` annotációt a gender mezőn. (Szüksége lehet a séma frissítésére – hbm2ddl -> update)

Haladó projekt:

1. Készítsünk DAO (Data Access Object) osztályt az Animal osztályhoz

a) Adjunk a projekthez egy új interfészt: AnimalDAO

- Az interfész terjessze ki az AutoCloseable interfészt
- Az alábbi metódusokat kell tartalmaznia:

```
public void saveAnimal(Animal a);  
public void deleteAnimal(Animal a);  
public void updateAnimal(Animal a);  
public List<Animal> getAnimals();
```

b) Adjuk meg az AnimalDAO egy implementációját a JpaAnimalDAO osztályban

- Tegyük ebbe az osztályba az EntityManagerFactory-t és az EntityManager-t példányosító sorokat az Application osztályból
- Zárjuk be mindkettőt a close() metódusban

c) Implementáljuk az egyszerű adatkezelő metódusokat a megfelelő helyeken az Application osztályban látottak szerint.

- d) Az össze állat lekérdezésére szolgáló metódus kódja az alábbi JPQL lekérdezéssel implementálható:

```
TypedQuery<Animal> query = entityManager.createQuery(
    "SELECT a FROM Animal a", Animal.class);

List<Animal> animals = query.getResultList();

return animals;
```

2. Módosítsuk a main metódust úgy, hogy a DAO-nkat használja az adatbázis eléréséhez.
- a) Használjuk a DAO automatikus bezárására a try-with-resources struktúrát

Haladó projekt 2 – Kollekciónak leképezése:

1. Adjunk egy Zoo osztályt a projekthez. Egy zoo objektumnak az alábbi mezői vannak: id, name, animals. utóbbi egy Set mely az állatkertben élő állatokat tartalmazza. (IntelliJ IDEA esetében adjunk új @OneToMany típusú elemet az entitáshoz)
2. Készítsünk a Zoo osztályból entitást annotációkkal az alábbiak szerint:

```
@OneToMany(cascade = CascadeType.ALL)
@JoinColumn(name = "zoo_id")
private Set<Animal> animals;
```

3. A main metódusban példányosítsunk egy Zoo objektumot és adjunk hozzá állatokat. Figyeljük meg az adatok változását az adatbázisban próba közben.
4. Opcionálisan egészítsük ki az AnimalDAO interfészt egy saveZoo(Zoo zoo) metódussal, hogy ezt is menthessük a DAO-n át.

5. Töröljük a Zoo osztályból az animals attribútumot és kapcsolt metódusait.
6. Adjunk az Animal osztályhoz egy új mezőt owner_zoo néven. A típusa legyen Zoo. (IntelliJ IDEA esetében adjunk új @ManyToOne típusú elemet az entitáshoz)
7. Használjuk rajta az alábbi annotációkat:

```
@ManyToOne
@Cascade(CascadeType.ALL)
@JoinColumn(name = "zoo_id")
private Zoo owner_zoo;
```

8. Példányosítsunk egy Animal-t és állítsuk be az owner_zoo értékét egy új állatkertre. Perzisztáljuk, majd nézzük meg az adatbázis változásait.